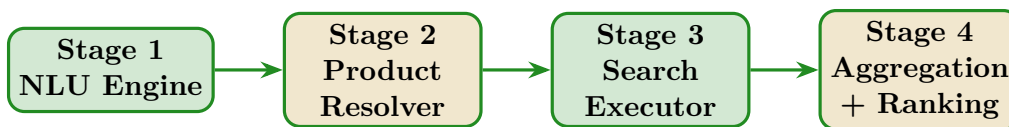

ZaraiLink Search Engine Infrastructure

Technical Architecture & Design Report



Project: ZaraiLink — Agricultural Trade Intelligence Platform
Component: Search & Discovery Engine
Stack: Django 4.2, PostgreSQL, Redis, LightGBM, SentenceTransformers
Version: FYP Final Report
Date: April 27, 2026

Contents

1	Executive Summary	3
2	System Architecture Overview	4
2.1	High-Level Component Map	4
2.2	Technology Stack	4
3	Stage 1: Natural Language Understanding Engine	5
3.1	Overview	5
3.2	Fast Path: HS Code Detection	5
3.3	Intent Detection Pipeline	5
3.4	LLM Unified Extraction	5
3.5	Product Keyword Resolution	6
3.6	Country Identification	6
3.7	Scope Inference	6
3.8	NLU Output Schema	7
3.9	NLU Caching	7
4	Stage 2: Product Resolution	8
4.1	Overview	8
4.2	Five-Pass Matching Cascade	8
4.3	HS Code Resolution	8
4.4	Scope-Aware Trade Type Mapping	8
4.5	Availability Check Cache	9
5	Stage 3: Search Execution	10
5.1	Three-Path Execution Strategy	10
5.2	Path A: ORM Aggregation (Active)	10
5.3	Path B: OpenSearch Hybrid (Dormant)	11
5.4	Path C: Company-Name Fallback	11
5.5	Search Request Parameters	11
6	Stage 4: Aggregation, Intelligence & Ranking	12
6.1	SupplierAggregator	12
6.2	Intelligence Metrics	12
6.3	Volume-Fit Scoring	12
6.4	Intent-Aware Composite Ranking	13
6.5	Aggregation Cache	13
7	API Endpoints	14
7.1	Main Search — GET /api/search/	14
7.2	Autocomplete — GET /api/search/autocomplete/	14
7.3	Supplier Detail — GET /api/search/supplier-detail/	15
7.4	Supplier Compare — GET /api/search/compare/	15
8	Caching Architecture	16
8.1	Cache Backend Selection	16
8.2	Cache Hierarchy	16
8.3	Cache Key Construction	16

9	Machine Learning Models	18
9.1	Model Inventory	19
9.2	SetFit Intent Classifier	20
9.3	LightGBM LambdaRank	20
9.4	KeyBERT Keyword Extraction	20
10	Resilience & Fallback Design	21
11	Logging & Observability	22
11.1	Logger Configuration	22
11.2	Log Namespaces	22
12	Database Schema Interaction	23
12.1	Core Tables Used	23
12.2	PostgreSQL Extensions Required	23
13	End-to-End Query Walkthrough	24
13.1	Stage 1: NLU	24
13.2	Stage 2: Product Resolution	24
13.3	Stage 3: Search Execution	24
13.4	Stage 4: Aggregation & Ranking	25
13.5	Response	25
14	Source Code Structure	26
15	Configuration & Deployment Notes	27
15.1	Environment Variables	27
15.2	Development Setup	27
15.3	Required PostgreSQL Extension	27
15.4	Enabling OpenSearch	27
16	Key Design Decisions	29
17	Future Work	29
18	Conclusion	30

1 Executive Summary

ZaraiLink is a Pakistani agricultural trade intelligence SaaS platform that aggregates real government import/export transaction records and exposes them through a specialised natural-language search interface. The search engine is the platform’s core value proposition: it lets traders express intent in plain Urdu-inflected English (*“cheapest sugar from Brazil”, “buyers for rice bulk 1000 mt”*) and receive a ranked list of verified trading partners drawn directly from customs data.

The engine is designed around four sequential stages:

1. **NLU Engine** — Extracts structured intent, products, country, price and quantity signals from a raw free-text query using a hybrid of regex rules, fine-tuned ML classifiers and a large-language-model (LLM) fast-path.
2. **Product Resolution** — Maps the extracted product keyword to one or more `ProductSubCategory` database identifiers through a five-pass cascade of exact, substring, trigram and edit-distance matching.
3. **Search Execution** — Queries the PostgreSQL database via Django ORM aggregation (with a dormant OpenSearch/BM25+kNN path available for future activation).
4. **Aggregation & Ranking** — Computes per-company trade statistics, intelligence signals and an intent-aware composite relevance score using LightGBM LambdaRank.

The system is built for resilience: every external dependency (LLM APIs, Redis, OpenSearch) has a deterministic fallback so that search always returns results even under partial infrastructure failure.

2 System Architecture Overview

2.1 High-Level Component Map

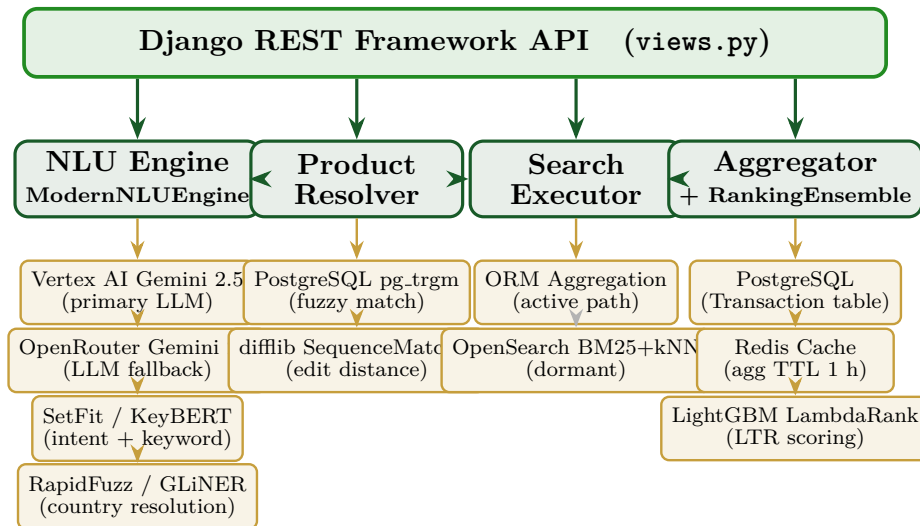


Figure 1: ZaraiLink search engine component map. Solid gold arrows show active dependencies; dashed arrow indicates a dormant path.

2.2 Technology Stack

Layer	Technology	Role
Web framework	Django 4.2 + Django REST Framework	API endpoints, ORM
Database	PostgreSQL (pg_trgm extension)	Primary data store
Cache	Redis 7 / LocMemCache (fallback)	NLU & aggregation cache
LLM primary	Vertex AI Gemini 2.5 Flash Lite	Unified entity extraction
LLM fallback	OpenRouter Gemini 2.0 Flash Lite	LLM resilience
Intent classifier	SetFit (HuggingFace)	Buy/Sell classification
Keyword extractor	KeyBERT (paraphrase-MiniLM-L6-v2)	Product keyword fallback
NER model	GLiNER (urchade/gliner_base)	Zero-shot NER (optional)
Fuzzy matching	RapidFuzz	Country name resolution
Ranker	LightGBM LambdaRank	LTR scoring
Semantic search	all-MiniLM-L6-v2 (FAISS HNSW)	OpenSearch kNN (dormant)

Table 1: Technology stack by layer

3 Stage 1: Natural Language Understanding Engine

3.1 Overview

The `ModernNLUEngine` class (file: `backend/search/services/nlu_engine.py`, 1,516 lines) converts a raw free-text query into a machine-readable intent structure. It uses a *hybrid* extraction strategy: cheap deterministic rules run first, an LLM is consulted for the hard cases, and every component has a keyword-based or heuristic fallback so parsing *never blocks*.

3.2 Fast Path: HS Code Detection

Before any ML inference, the engine checks whether the query is a raw HS (Harmonised System) tariff code. Supported formats include dot-notation (1702.3090), spaced (1702 3090) and hyphen-separated (17-02-30-90). On a match the query is normalised to canonical dot form and the full NLU pipeline is bypassed.

3.3 Intent Detection Pipeline

Intent classification determines whether a user is looking to **buy** (find suppliers) or **sell** (find buyers). The pipeline runs three sub-steps in priority order:

1. **Semantic reversal rules** — 182 longest-match phrase patterns that handle constructs such as “*looking for sellers of X*” $\Rightarrow$ **BUY**, or “*we export X*” $\Rightarrow$ **SELL**.
2. **SetFit classifier** — A SetFit model fine-tuned on 30 trade-domain labelled examples (`backend/search/models/intent_model/`). SetFit requires very few labelled examples due to contrastive sentence-pair pre-training.
3. **Keyword regex fallback** — Matches explicit cue words (*buy, purchase, import, supplier, source* $\Rightarrow$ **BUY**; *sell, export, buyer, customer* $\Rightarrow$ **SELL**). If no cue is found the engine defaults to **BUY**, the more common query type for Pakistani importers.

3.4 LLM Unified Extraction

A single LLM call (max 256 tokens) extracts the following entities simultaneously:

- **Products** — up to two product keywords as an array
- **Country** — origin or destination country
- **Price signal** — operator (`lte/gte`), numeric value, currency, and a ranking hint (`price_asc / price_desc`)
- **Quantity** — volume in metric tonnes if mentioned

Primary LLM: Vertex AI `gemini-2.5-flash-lite` (0.29s time-to-first-token).

Fallback LLM: OpenRouter `google/gemini-2.0-flash-lite` — activated automatically on Vertex API error, quota exhaustion or timeout.

3.5 Product Keyword Resolution

After the LLM call the extracted product string is further resolved via a four-step cascade:

1. **LLM output** — used directly if a valid non-empty string.
2. **KeyBERT** — extracts the highest-scoring n-gram from the query using the sentence-transformer `paraphrase-MiniLM-L6-v2` with a 240-word stop list tuned for agricultural trade jargon.
3. **Trigram similarity** — if neither of the above yields a result, the query is matched directly against PostgreSQL's `pg_trgm` index on `ProductSubCategory.name` (threshold 0.3).
4. **Stop-word strip** — strip all known stop phrases and take the longest remaining token.

3.6 Country Identification

Country resolution uses three layers:

1. **Dictionary lookup** — 80+ countries in a hand-curated mapping keyed on common variants, abbreviations and Urdu-romanised names.
2. **LLM cross-validation** — the country returned by the LLM is reconciled with the dictionary to handle aliases.
3. **RapidFuzz WRatio** — fuzzy matching against the dictionary keys with an 80% confidence threshold to handle OCR errors and non-standard spellings.
4. **GLiNER NER** (optional, disabled by default) — zero-shot span extraction using `urchade/gliner_base` can be enabled with the `_GLINER_ENABLED` flag.

3.7 Scope Inference

The *scope* signals whether the user wants Pakistani trading partners only or worldwide results:

Signal	Inferred Scope
A non-Pakistani foreign country resolved	worldwide
Local city / Pakistani domain key-word	pakistan
No geographic signal	Inherit from UI context

Table 2: Scope inference rules

3.8 NLU Output Schema

The NLU engine returns a Python dict with the following guaranteed keys:

```

1 {
2     "intent":          "BUY" | "SELL",
3     "product":         str,          # primary resolved
4         keyword
5     "product_keyword": str,          # alias (same value)
6     "product_keyword2": str,        # secondary product if
7         two mentioned
8     "country":         str | None,
9     "quantity":        str | None,  # e.g. "1000 MT"
10    "price_filter": {
11        "range": {"usd_per_mt": {"lte": "gte": float}},
12        "ranking_hint": "price_asc" | "price_desc" | None
13    },
14    "os_filter":        {...},        # OpenSearch bool filter
15        clauses
16    "entities":         [...],        # GLiNER span extractions
17    "ui_context":       "worldwide" | "pakistan",
18    "is_hs_query":      bool
19 }
```

Listing 1: NLU output schema (Python)

3.9 NLU Caching

- **Cache key:** "nlu_" + MD5(query.lower() + ":" + scope.lower())
- **TTL:** 24 hours
- **Backend:** Redis if reachable; LocMemCache otherwise.

4 Stage 2: Product Resolution

4.1 Overview

Product resolution maps the NLU-extracted product keyword to a list of `ProductSubCategory` primary keys, which are the anchors for all downstream database queries. The resolver also handles HS-code queries and multi-variant products.

4.2 Five-Pass Matching Cascade

Pass	Method	Logic	Threshold
1	Prefix (SubCategory)	<code>ProductSubCategory.name</code> <code>__startswith</code>	exact
2	Prefix (Item)	<code>ProductItem.name</code> <code>__startswith</code>	exact
3	Substring	<code>icontains</code> on both tables	exact
4	Trigram fuzzy	PostgreSQL <code>TrigramSimilarity</code>	> 0.30
5	Edit distance	Python <code>difflib.SequenceMatcher</code>	≥ 0.75
6	Needle-in-haystack	Word-by-word token search for garbled queries	any token

Table 3: Product resolution pass cascade

4.3 HS Code Resolution

When the query is flagged as an HS-code query the resolver performs a cascade lookup against `ProductSubCategory.hs_code`:

1. Exact match on normalised code (e.g. 1702.3090)
2. Prefix match (`startswith`) to capture six-, four- and two-digit chapter codes
3. Progressively shorter prefix to handle partial codes

If more than **20** subcategory variants match the same chapter code, the engine sets `needs_disambiguation = True` and returns a variant picker payload to the frontend rather than a result list.

4.4 Scope-Aware Trade Type Mapping

To prevent zero-result pages the resolver maps (*intent*, *scope*) to a `trade_type` filter, since Pakistani customs data records each transaction from the Pakistani importer’s perspective:

Intent	Scope	Trade Type	Rationale
BUY	pakistan	EXPORT	Pakistani sellers exporting
SELL	pakistan	IMPORT	Pakistani buyers importing
BUY	worldwide	IMPORT	Foreign suppliers delivering to Pak
SELL	worldwide	EXPORT	Foreign buyers receiving from Pak

Table 4: Intent $\times$ scope $\rightarrow$ trade type mapping

4.5 Availability Check Cache

Before executing the full aggregation query, a lightweight availability check is run to confirm that at least one transaction exists for the resolved subcategories under the selected trade type.

- **Cache key:** "avail_" + MD5(subcat_ids:trade_type:country)
- **TTL:** 1 hour

5 Stage 3: Search Execution

5.1 Three-Path Execution Strategy

The search executor chooses one of three query paths based on service availability:

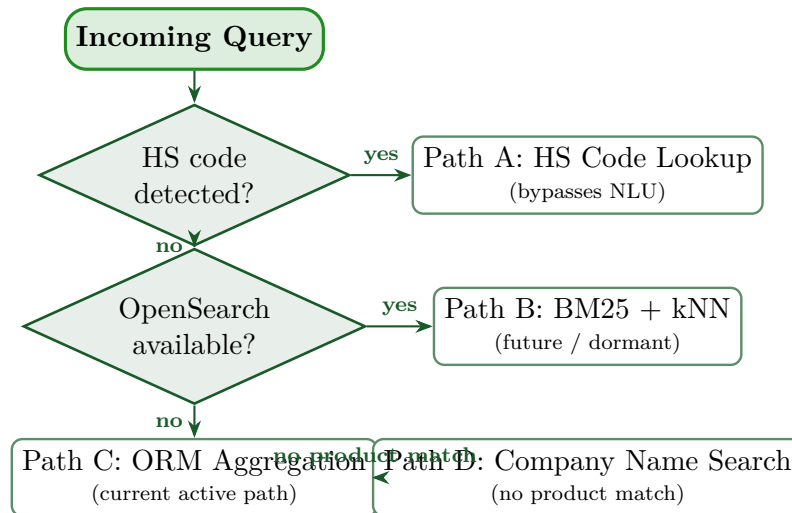


Figure 2: Search execution path selection. The current active path is Path C (ORM aggregation). Path B (OpenSearch) is fully implemented but dormant pending cluster setup.

5.2 Path A: ORM Aggregation (Active)

The currently active path aggregates the Transaction table using Django ORM with the following pipeline:

```

1 Transaction.objects
2   .filter(
3       sub_category_id__in=subcat_ids,
4       trade_type=trade_type,
5       **({"origin_country__icontains": country} if country else {}),
6   )
7   .values("seller" if intent=="BUY" else "buyer")
8   .annotate(
9       total_volume=Sum("qty_mt"),
10      avg_price=Avg("usd_per_mt"),
11      shipment_count=Count("id"),
12      last_shipment_date=Max("reporting_date"),
13  )
14  .order_by("-total_volume")

```

Listing 2: Core ORM aggregation query structure (simplified)

Key design decisions:

- Grouping is by **company name** rather than a surrogate key because the same physical company may appear under slightly varying spellings across transaction records.
- A **post-aggregation price filter** (equivalent to an SQL `HAVING` clause) applies the NLU price constraint after the `avg_price` annotation has been computed.

5.3 Path B: OpenSearch Hybrid (Dormant)

The OpenSearch path is fully implemented but currently disabled via a compile-time flag in `search_service.py` (line 232). When enabled it issues a hybrid query:

- **BM25 text search** with per-field boosts: `clean_product_name`×9, `sub_category_name`×7, `product_item_name`×5, `buyer/seller`×1.
- **kNN dense retrieval** using 384-dimensional sentence embeddings (`all-MiniLM-L6-v2`), indexed as FAISS HNSW in the `trade_index` index.
- Scores are fused via OpenSearch’s native `bool.should` with tunable weights.

The index is populated by the management command `python manage.py sync_search` (344 lines, `sync_search.py`).

5.4 Path C: Company-Name Fallback

When the five-pass product resolver returns no subcategory matches *and* the stripped query token is at least three characters long, the executor performs a direct `icontains` search against `Transaction.seller` and `Transaction.buyer`. Duplicate company names are deduplicated and returned as lightweight company profiles.

5.5 Search Request Parameters

Parameter	Type	Description
<code>q</code>	string	Raw natural-language query
<code>scope</code>	enum	<code>worldwide</code> or <code>pakistan</code> ; UI context
<code>hs_code</code>	string	Optional direct HS code lookup, bypasses NLU
<code>subcat_id</code>	integer	Optional resolved subcategory ID, bypasses resolution
<code>variant_name</code>	string	Optional exact product variant name filter

Table 5: GET `/api/search/` request parameters

6 Stage 4: Aggregation, Intelligence & Ranking

6.1 SupplierAggregator

The `SupplierAggregator` class (`backend/search/services/aggregation.py`, 654 lines) provides three public methods:

`get_suppliers_for_subcategories()` Main search result aggregation. Groups transactions by company, computes volume/price/recency, applies volume-fit scoring if a quantity signal is present.

`get_supplier_details()` Deep-dive for a single seller. Returns monthly sparklines, shipment-size distribution, buyer insights and a full transaction history (top 50 records).

`get_buyer_details()` Symmetrical to supplier details for the SELL intent flow. Shows purchase patterns, supplier relationships and order-size distribution.

6.2 Intelligence Metrics

The `_calculate_intelligence()` method computes four dynamic signals from the raw transaction data:

Signal	Definition
Repeat Ratio	Fraction of transactions involving counterparties that appear more than once. Indicates relationship stability.
Concentration Ratio	Percentage of total volume accounted for by the top-3 counterparties. High concentration = single-customer risk.
Pricing Label	Categorical label derived from average unit price vs. market median. Labels: <i>Premium</i> / <i>Competitive</i> / <i>Stable</i> (seller); <i>Opportunistic</i> / <i>Sensitive</i> / <i>Stable</i> (buyer).
Momentum Label	Volume trend direction vs. dataset maximum date. Labels: <i>Growing</i> / <i>Stable</i> / <i>Declining</i> .
Generated Summary	Human-readable one-sentence insight synthesising the above signals.

Table 6: Per-company intelligence signals

6.3 Volume-Fit Scoring

When the query contains a quantity signal (e.g. “1000 MT of sugar”), each company is scored on its ability to fulfil that order:

$$s_{\text{vol}} = 0.5 \cdot \min\left(\frac{V_{\text{max}}}{Q}, 1\right) + 0.3 \cdot \min\left(\frac{V_{\text{total}}}{Q}, 1\right) + 0.2 \cdot \min\left(\frac{\bar{V}}{Q}, 1\right) \quad (1)$$

where V_{max} is the largest single shipment, V_{total} the company's cumulative volume, $\bar{V}$ the average shipment size, and Q the requested quantity. The score is binned into four labels:

s_{vol} range	Label
≥ 0.80	<i>Strong</i>
≥ 0.50	<i>Good</i>
≥ 0.30	<i>Partial</i>
< 0.30	<i>Low</i>

6.4 Intent-Aware Composite Ranking

Each company receives a composite relevance score computed from four normalised signals:

$$\text{score} = w_1 \hat{V} + w_2 \hat{C} + w_3 \hat{P} + w_4 \hat{R} \quad (2)$$

where $\hat{V}$, $\hat{C}$, $\hat{P}$, $\hat{R}$ are min-max normalised values of *total volume*, *shipment count*, *inverse unit price* (lower price scores higher) and *recency date* respectively.

The weight vector is selected by the NLU `ranking_hint`:

Hint	w_1 (vol)	w_2 (count)	w_3 (price)	w_4 (recency)	Trigger
<code>price_asc</code>	0.15	0.15	0.50	0.20	cheap, affordable, budget
<code>price_desc</code>	0.45	0.25	0.05	0.25	bulk, premium, large order
<code>default</code>	0.35	0.25	0.15	0.25	no price signal

Table 7: Composite ranking weight presets

Price handling edge case: when `avg_price` is zero (unknown) and the hint is *not price_asc*, the price term contributes a neutral $w_3 \times 0.5$ rather than zero to avoid penalising companies with missing price data.

6.5 Aggregation Cache

- **Cache key:** "agg-" + MD5(subcat_ids:intent:scope:country:price:product_items)
- **TTL:** 1 hour
- **Backend:** Redis / LocMemCache fallback

7 API Endpoints

All endpoints are served under the `/api/search/` prefix and use Django REST Framework's JSON renderer. Authentication is session-based.

7.1 Main Search — GET `/api/search/`

URL: GET `/api/search/?q={query}&scope={scope}`

Response schema:

```

1  {
2      "query":                "<original user input>",
3      "parsed_query":        { /* full NLU result */ },
4      "needs_disambiguation": false,
5      "is_broad_search":     false,
6      "variants":            [],
7      "results": [
8          {
9              "name":          "Ningbo Trading Co.",
10             "country":       "China",
11             "total_volume":   12500.5,        // metric tonnes
12             "avg_price":      650.25,        // USD per MT
13             "shipment_count": 45,
14             "last_shipment_date": "2023-12-15",
15             "avg_shipment_vol": 277.8,        // MT per
16                                     shipment
17             "type":           "Supplier",    // or "Buyer"
18             "relevance_score": 0.8742,
19             "volume_fit":     "Strong"       // or N/A
20         },
21     ],
22     "market_snapshot": {
23         "total_count":        12,
24         "avg_price_global":   645.33,    // volume-weighted market
25                                     average
26         "top_country":       "China"
27     },
28     "count":                  12,
29     "search_engine": "orm"        // or "opensearch"
30 }
```

Listing 3: Search response schema

7.2 Autocomplete — GET `/api/search/autocomplete/`

URL: GET /api/search/autocomplete/?q={prefix}

Returns up to 8 ProductSubCategory suggestions ranked by total trade volume, with labels displaying volume in metric tonnes.

7.3 Supplier Detail — GET /api/search/supplier-detail/

URL: GET /api/search/supplier-detail/?name={company}&query={q}&scope={scope}

Returns full deep-dive including monthly volume/price sparklines, shipment-size bucket chart, unique buyer count, top transactions (up to 50) and intelligence signals.

7.4 Supplier Compare — GET /api/search/compare/

URL: GET /api/search/compare/?suppliers=A,B,C&query={q}

Returns a side-by-side comparison object for up to three companies, including price ranges, volume sparklines, shipment date ranges and destination country breakdowns.

8 Caching Architecture

8.1 Cache Backend Selection

The platform uses a two-tier fallback strategy for its cache backend:

1. **Redis** — preferred backend, auto-detected on `127.0.0.1:6379`. If a connection can be established at startup, the Django cache backend is set to `django-redis`.
2. **LocMemCache** — in-process Python dict used when Redis is unavailable. Not shared between processes and not persistent across restarts, but guarantees that caching *never fails*.

8.2 Cache Hierarchy

Cache Name	Key Prefix	Covers	TTL
NLU cache	nlu_	Parsed intent struct for a given (query, scope) pair	24 h
Availability check	avail_	Whether any transactions exist for (subcat_ids, trade_type, country)	1 h
Aggregation cache	agg_	Full ranked supplier/buyer list for a given parameter set	1 h

Table 8: Cache tiers and TTLs

8.3 Cache Key Construction

All cache keys are deterministic MD5 hashes of the relevant parameter tuple, making them collision-resistant and length-bounded:

```

1  import hashlib
2
3  def _make_key(prefix: str, *parts) -> str:
4      raw = ":".join(str(p) for p in parts)
5      return prefix + hashlib.md5(raw.encode()).hexdigest()
6
7  # Examples
8  nlu_key = _make_key("nlu_", query.lower(), scope.lower())
9  agg_key = _make_key("agg_", subcat_ids, intent, scope,
10                      country, price, product_items)
11  avail_key = _make_key("avail_", subcat_ids, trade_type, country
12                        )

```

Listing 4: Cache key construction pattern

9 Machine Learning Models

9.1 Model Inventory

Model	Type	Role	Status
SetFit intent classifier	Few-shot contrastive fine-tune	Buy/Sell intent classification (30 examples)	Active
KeyBERT (paraphrase-MiniLM-L6-v2)	Sentence-transformer + BERT	Product keyword extraction fallback	Active
GLiNER (urchade/gliner_base)	Zero-shot NER	Country/entity span extraction	Disabled
LightGBM LambdaRank	Gradient-boosted trees	LTR re-ranking of aggregated results	Active
all-MiniLM-L6-v2 (FAISS HNSW)	Sentence-transformer	384-dim dense retrieval index	Dormant
Vertex AI Gemini 2.5 Flash Lite	LLM (API)	Unified entity extraction	Active
OpenRouter Gemini 2.0 Flash Lite	LLM (API)	LLM fallback	Active (fallback)

Table 9: ML model inventory

9.2 SetFit Intent Classifier

SetFit (*Sentence Fine-Tuning*) is a parameter-efficient classification paradigm that first fine-tunes a sentence-transformer via contrastive learning on a small set of labelled pairs, then trains a lightweight head on the resulting embeddings.

- **Training set:** 30 trade-domain sentences per class (BUY / SELL)
- **Model artefact:** `backend/search/models/intent_model/`
- **Inference latency:** < 5 ms (CPU)

9.3 LightGBM LambdaRank

The LTR model (`backend/search/models/lgbm_ltr.txt`) provides a learned relevance signal used to supplement the heuristic composite score.

- **Algorithm:** LambdaRank (listwise, NDCG-optimised)
- **Features:** normalised volume, shipment count, recency, price signal, country match, volume-fit score
- **Integration:** 30% weight in the final ensemble ($0.7 \times \text{heuristic} + 0.3 \times \text{LightGBM}$)

9.4 KeyBERT Keyword Extraction

KeyBERT uses the sentence-transformer `paraphrase-MiniLM-L6-v2` to score candidate n-grams by cosine similarity to the full query embedding. A 240-word stop list filters out supply-chain meta-language (“importer”, “looking for”, “cheap”, “kg”) so the model focuses on the product noun phrase.

10 Resilience & Fallback Design

A core design principle of the ZaraiLink search engine is that **no single component failure should return an error to the user**. Every external dependency has a deterministic fallback:

Component	Failure Mode	Fallback
Vertex AI (LLM primary)	API error, quota, timeout	OpenRouter LLM
OpenRouter (LLM fallback)	API error	KeyBERT + regex + dict
SetFit classifier	Model file missing	Keyword regex rules
KeyBERT	Model file missing	Stop-word strip heuristic
GLiNER NER	Disabled by default	Dict lookup + RapidFuzz
Redis cache	Connection refused	LocMemCache (in-process)
OpenSearch	Disabled / unreachable	ORM aggregation path
Product resolver (no match)	Zero subcategory matches	Company-name icontains search

Table 10: Failure modes and fallbacks

The OpenSearch client is configured with a **3-second timeout** to ensure fast failure in case the cluster is temporarily unresponsive. The ORM fallback is triggered silently; the response includes a `"search_engine": "orm"` field to assist debugging.

11 Logging & Observability

11.1 Logger Configuration

The `search` logger is configured at `INFO` level with a console (`stdout`) handler. No structured logging aggregator is connected in the current deployment; logs are captured by the hosting platform's `stdout` stream.

11.2 Log Namespaces

Prefix	Events logged
[NLU]	Intent, product keyword, country, price extracted per query
[TIMING NLU]	Per-subsystem latency (LLM call, KeyBERT, fuzzy matching)
[CACHE]	Hit / miss notifications with key prefix
[Ranking]	Weight preset selected (<code>price_asc</code> / <code>price_desc</code> / default)
[Search]	Query dispatched, path selected (ORM / OpenSearch / company fallback)

Table 11: Log namespace reference

12 Database Schema Interaction

12.1 Core Tables Used

Table (Django model)	Key Columns	Used by
Transaction	seller, buyer, qty_mt, usd_per_mt, trade_type, reporting_date, origin_country, destination_country, hs_code, tx_reference	Stages 3, 4
ProductSubCategory	name, hs_code, category_id	Stages 1, 2
ProductItem	name, sub_category_id	Stage 2

Table 12: Database tables accessed by the search engine

12.2 PostgreSQL Extensions Required

The product resolver’s trigram fuzzy matching requires the `pg_trgm` extension:

```

1  -- Run once as superuser
2  CREATE EXTENSION IF NOT EXISTS pg_trgm;

```

Listing 5: `pg_trgm` installation

The extension enables `TrigramSimilarity` and `GinIndex` on `ProductSubCategory.name` and `ProductItem.name`, which are exploited by Django’s `TrigramSimilarity` database function in pass 4 of the resolver.

13 End-to-End Query Walkthrough

This section traces a representative query through all four pipeline stages.

Example query: *“I want to buy cheap refined sugar under \$500 from Brazil”*
UI context: worldwide

13.1 Stage 1: NLU

1. **HS fast path:** No HS code pattern found.
2. **Semantic reversal:** “I want to buy” matches BUY pattern. SetFit confirms BUY with high confidence.
3. **LLM unified extraction:** products=["refined sugar"], country="Brazil", price={operator:"lte", value:500, currency:"USD", ranking_hint:"price_asc"}.
4. **Scope inference:** country="Brazil" $\notin$ Pakistan set $\Rightarrow$ scope="worldwide".
5. **NLU output:** intent=BUY, product="refined sugar", country="Brazil", price_filter.range={usd_per_mt:{lte:500}}, ranking_hint="price_asc".

13.2 Stage 2: Product Resolution

1. Pass 1 (prefix): “refined sugar” matches ProductSubCategory names starting with “refined sugar” $\Rightarrow$ IDs {42, 44, 47}.
2. Trade type mapping: BUY $\times$ worldwide $\Rightarrow$ trade_type=IMPORT.
3. Availability check: cache miss; query confirms transactions exist.

13.3 Stage 3: Search Execution

ORM path executes:

```

1  SELECT  seller ,
2          SUM(qty_mt)           AS total_volume ,
3          AVG(usd_per_mt)       AS avg_price ,
4          COUNT(*)              AS shipment_count ,
5          MAX(reporting_date) AS last_shipment_date
6  FROM    trade_data_transaction
7  WHERE   sub_category_id IN (42, 44, 47)
8          AND trade_type = 'IMPORT'
9          AND origin_country ILIKE '%Brazil%'
10 GROUP BY seller
11 HAVING  AVG(usd_per_mt) <= 500    -- post-aggregation price
         filter

```



```
12 ORDER BY total_volume DESC;
```

Listing 6: Effective SQL (paraphrased)

13.4 Stage 4: Aggregation & Ranking

1. **Weight preset:** `ranking_hint="price_asc"` selects $(w_1, w_2, w_3, w_4) = (0.15, 0.15, 0.50, 0.20)$.
2. **Min-max normalisation:** applied per signal across all returned rows.
3. **Score:** $\text{score} = 0.15\hat{V} + 0.15\hat{C} + 0.50\hat{P} + 0.20\hat{R}$ where $\hat{P} = \widehat{1/\text{price}}$ (inverted so cheaper = higher score).
4. **Output:** Ranked list of Brazilian refined-sugar exporters, sorted by composite score descending. Top result will be the cheapest, most frequent, most recently active supplier.

13.5 Response

```
1 {
2   "query": "I want to buy cheap refined sugar under $500 from
3     Brazil",
4   "parsed_query": {
5     "intent": "BUY", "product": "refined sugar",
6     "country": "Brazil", "ui_context": "worldwide",
7     "price_filter": {"range": {"usd_per_mt": {"lte": 500}}},
8     "ranking_hint": "price_asc"
9   },
10  "results": [
11    {"name": "Sao Paulo Acucar Ltda.", "country": "Brazil",
12     "total_volume": 8400.0, "avg_price": 412.50,
13     "shipment_count": 28, "relevance_score": 0.921}
14  ],
15  "market_snapshot": {"avg_price_global": 423.8, "top_country": "Brazil"},
16  "count": 7, "search_engine": "orm"
}
```

Listing 7: Abbreviated API response

14 Source Code Structure

File	Lines	Responsibility
search/services/nlu_engine.py	1,516	Full NLU with 5-stage extraction, LLM integration, caching
search/services/search_service.py	945	Pipeline orchestration, product resolver, execution paths, ranking
search/services/aggregation.py	654	ORM aggregation, intelligence scoring, volume-fit, sparklines
search/views.py	271	DRF API endpoints, request parsing, response serialisation
search/management/commands/sync_search.py	344	OpenSearch index builder (management command)
search/urls.py	29	URL routing for all search endpoints

Table 13: Search module source files

Model Artefact	Description
search/models/intent_model/	SetFit fine-tuned checkpoint (HuggingFace format)
search/models/lgbm_ltr.txt	LightGBM LambdaRank trained model (text format)
search_index.pkl	Pickled in-memory search index (alternative to OpenSearch)

Table 14: Model artefacts

15 Configuration & Deployment Notes

15.1 Environment Variables

Variable	Purpose
OPENAI_API_KEY	OpenAI API key (text-embedding-3-small, gpt-4o-mini)
GOOGLE_APPLICATION_CREDENTIALS	Path to service account JSON for Vertex AI Gemini
OPENROUTER_API_KEY	OpenRouter API key (LLM fallback)
REDIS_URL	Redis connection string; defaults to <code>redis://127.0.0.1:6379/1</code>
OPENSEARCH_HOST	OpenSearch cluster host (optional)
DEBUG	Django debug flag; hardcoded <code>True</code> in dev

Table 15: Relevant environment variables

15.2 Development Setup

- Backend: `http://localhost:8000` (Django dev server)
- Frontend: `http://localhost:3000` (Create React App)
- Database: PostgreSQL local, `user=postgres`, `db=zarailink`
- Redis: optional, auto-detected on `127.0.0.1:6379`

15.3 Required PostgreSQL Extension

```
psql -U postgres -d zarailink -c "CREATE EXTENSION IF NOT EXISTS pg_trgm;"
```

15.4 Enabling OpenSearch

To activate the OpenSearch hybrid path:

1. Deploy an OpenSearch cluster and set `OPENSEARCH_HOST`.
2. Run `python manage.py sync_search` to build the `trade_index`.
3. Change the compile-time flag on line 232 of `search_service.py`:

```
1 # search_service.py, line 232  
2 USE_OPENSEARCH = True    # was False
```

Listing 8: Enabling OpenSearch path

16 Key Design Decisions

1. **Single LLM call for all entities.** Rather than making separate API calls for product, country and price extraction, a single structured prompt extracts all three in one round trip (max 256 tokens). This keeps per-query LLM cost under \$0.0002 on Gemini 2.5 Flash Lite.
2. **Group by company name, not ID.** Pakistani customs data does not have stable company identifiers across datasets; grouping by normalised company name avoids artificial fragmentation of the same trading entity.
3. **Intent $\times$ scope $\rightarrow$ trade type mapping.** Explicitly mapping intent to the correct `trade_type` filter is the single most important correctness decision: querying the wrong trade type silently returns zero results.
4. **Post-aggregation price filter.** Applying the price constraint as a `HAVING`-equivalent after `AVG(usd_per_mt)` is computed means the filter operates on a representative average rather than on noisy individual transaction prices.
5. **Asymmetric weight presets.** Giving price a 50% weight for bargain queries and only 5% for bulk/premium queries reflects real buyer behaviour: cost-sensitive importers care first about price; volume traders care first about proven capacity.
6. **Graceful model degradation.** The SetFit, KeyBERT and GLiNER models are all loaded with try/except guards. If any model artefact is missing the system falls through to a faster, cheaper heuristic rather than raising an exception.

17 Future Work

1. **Activate OpenSearch hybrid path.** Enabling BM25+kNN retrieval would improve recall for long-tail agricultural product names that are not in the `ProductSubCategory` table.
2. **Enable GLiNER NER.** Zero-shot span extraction would improve country resolution for uncommon country names and compound geography expressions (e.g. “sub-Saharan Africa”).
3. **Structured logging + metrics.** Integrating OpenTelemetry would expose per-stage latency histograms and cache-hit ratios as Prometheus metrics, enabling SLO-based alerting.
4. **Continuous LTR retraining.** Collecting implicit feedback (click-through rate, contact-unlock events) and feeding it into periodic LightGBM retraining would improve ranking over time without manual labelling.
5. **Multilingual query support.** Adding Urdu-script query normalisation (Nastaliq to Latin transliteration) would serve traders who type in their native script.

18 Conclusion

The ZaraiLink search engine is a purpose-built agricultural trade discovery system that combines deterministic rule-based NLU, lightweight ML classifiers, LLM-powered entity extraction and ORM-level database aggregation into a resilient, sub-second query pipeline.

Its most important property is *correctness under degradation*: the system makes deliberate, conservative choices at every stage (defaulting to the BUY intent, applying the correct trade-type filter, falling back to ORM when distributed search is unavailable) to ensure that users always receive meaningful results even when individual infrastructure components are offline.

With the OpenSearch hybrid path and continuous LTR retraining activated, the platform is well-positioned to scale to larger product catalogues and higher query volumes while maintaining the contextual accuracy that differentiates it from generic keyword search.